

A Survey of KV-Cache Reuse and Hierarchical Memory Systems for Code Agents

Project Research Team
Independent Technical Survey
August 2026

Abstract—Code agents repeatedly invoke large language models while reading repositories, calling tools, editing files, and observing tests. Their prompts are long, multi-turn, and partially repetitive, making them promising workloads for cross-request key-value (KV) cache reuse. The opportunity is also easy to overstate: textual repetition does not imply reusable attention state, cache transfer can be slower than recomputation, and cross-tenant sharing can create timing and reconstruction risks. This survey synthesizes 76 primary papers and 18 official technical documents available through August 30, 2026. It develops a Code Agent-specific taxonomy spanning exact-prefix caching, prompt canonicalization, position-independent and partial reuse, hierarchical HBM/DRAM/NVMe or remote storage, prefill-decode disaggregation, compression and eviction, and architecture-aware representations for MQA, GQA, MLA, sparse, and hybrid models. It separates logical hits from token and profitable hits, derives correctness and benefit conditions, compares representative systems, and proposes a trace-driven evaluation protocol. The evidence supports exact-prefix reuse plus deterministic context construction as the first reproducible baseline. Non-prefix reuse, low-bit storage, and deep hierarchies are promising but require explicit quality, latency, compatibility, and isolation validation. No public evidence currently establishes the project’s proposed 95% Code Agent hit-rate, 90% hit-path TTFT reduction, or 10% cost-ratio targets; these remain hypotheses for measurement.

Index Terms—Code Agent, KV-cache, prefix caching, context reuse, hierarchical memory, LLM serving, time to first token, cache quantization

I. INTRODUCTION

Repository-level software engineering has become a representative agentic workload. SWE-bench asks a system to resolve real GitHub issues in executable repositories and explicitly exposes the need for multi-file reasoning, environment interaction, and long contexts [1]. SWE-agent and OpenHands operationalize the closed loop: the model reasons, calls repository or shell tools, observes results, modifies state, and invokes the model again [2], [3]. Agentless provides an important control because it replaces an open-ended agent loop with staged localization, repair, and validation [4]. The comparison shows that “Code Agent” denotes a system and workload, not merely a code-generating model.

Repeated calls in one task often retain system instructions, tool schemas, repository rules, earlier messages, and unchanged files while appending a command result, test failure, or patch. Recomputing the entire prompt prefill wastes compute. Exact-prefix caching, implemented in production engines and APIs, can skip computation for an identical token prefix [5], [6], [7], [8]. However, repository content is frequently reordered,

compacted, filtered, or locally edited. A byte-identical code span appearing after a different prefix or at a different position generally has different hidden and KV states. CacheBlend, EPIC, and recent position-independent methods therefore introduce correction or selective recomputation rather than concatenating arbitrary cached tensors [9], [10], [11].

The systems problem is equally important. A cache hit in host memory, SSD, or a remote pool is only useful when lookup, transfer, restoration, and scheduling overheads are lower than local recomputation. MemServe, Mooncake, LMCache, Strata, and HiCache illustrate the movement toward distributed or hierarchical KV storage [12], [13], [14], [15], [16]. Compression and eviction can expand effective capacity, but approximate state may change output quality, and compressed layouts may lack high-throughput kernels [17], [18], [19].

This survey addresses five research questions:

- 1) Which properties of Code Agent trajectories create safe reuse opportunities?
- 2) Under what execution identity and dependency conditions is cached KV reusable?
- 3) How do exact-prefix, non-prefix, hierarchical, and compressed methods differ?
- 4) Which metrics distinguish logical hits from performance and cost benefits?
- 5) Which research gaps must be closed before ambitious hit-rate and cost targets can be treated as evidence?

The contribution is not another generic catalog of KV compression. Existing reviews cover broad cache-consumption and acceleration techniques [20], [21]. Here, the organizing object is the Code Agent trajectory and the interaction between context construction, serving semantics, memory hierarchy, architecture, and security.

II. REVIEW METHOD AND EVIDENCE SCOPE

A. Corpus Construction

The survey used a structured search over arXiv, peer-reviewed proceedings, official project documentation, and references cited by seed systems. The cutoff date is August 30, 2026. Inclusion required a direct connection to at least one of five categories: (i) Code Agent workloads or repository benchmarks, (ii) prefix or position-independent reuse, (iii) hierarchical or disaggregated KV movement, (iv) compression, eviction, or attention architecture, and (v) evaluation or cache security. The final evidence set contains 76 papers and 18

TABLE I
EVIDENCE LEVELS AND PERMITTED INTERPRETATION.

Level	Source	Use and limitation
A	Peer-reviewed paper or formal proceedings	Core algorithm and system evidence; results remain configuration-specific.
B	Public preprint with methods and experiments	Emerging evidence; reproduction and later revision may be needed.
D	Official project or provider documentation	Current behavior, interfaces, metrics, and operational semantics.
P	Project target or internal hypothesis	Defines what to test; never reported as an achieved result.

official documents. Surveys were used for navigation; core claims were checked against primary papers or current official documentation.

Each item was assigned a project role and priority. Public availability was separated from redistribution permission: only papers with an explicit Creative Commons license were placed in the offline corpus. Other papers remain in the bibliographic index with direct official links. This distinction affects packaging, not scientific relevance.

B. Evidence Levels

Table I separates evidence types. A paper’s reported speedup is evidence for its own implementation and workload, not for an unseen Code Agent trace. Official documentation is authoritative for current interfaces but can change. Internal project targets are hypotheses until evaluated under a specified model, tokenizer, serving engine, hardware, concurrency, and cost model.

C. Threats to Literature Validity

Three biases matter. First, recent systems often use different models, GPUs, block sizes, datasets, and latency definitions. Second, many compression papers emphasize memory ratios without production kernels or end-to-end request latency. Third, public Code Agent benchmarks measure task success, not token-level prompt overlap, cache location, or transfer time. The survey therefore emphasizes mechanisms and testable conditions over headline speedups.

III. CODE AGENT WORKLOADS

A. Working Definition

A Code Agent is an LLM-centered software engineering system that can observe a repository and execution environment, select and invoke tools, modify project state, verify results, and iteratively decide the next action. The system usually contains an agent controller, context builder, model endpoint, tool interface, execution sandbox, and observation store. Claude Code’s official description similarly centers repository reading, file editing, command execution, and development-tool integration [22].

The model call at step t can be abstracted as

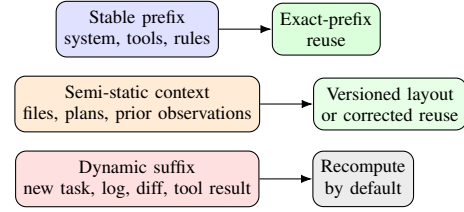


Fig. 1. Prompt stability classes imply different cache semantics.

$$P_t = S\|T\|R\|F_t\|H_t\|Q_t, \quad (1)$$

where S is the system instruction, T tool schemas, R repository rules, F_t selected files or code, H_t dialogue and tool history, and Q_t the current objective. After a tool call, H_{t+1} typically appends an observation, while F_t may change locally. Equation 1 is a workload abstraction, not a claim that every framework uses this exact ordering.

B. Workload Diversity

The SWE family now covers static, live, multilingual, multimodal, training, exploration, and long-horizon variants [23], [24], [25], [26], [27], [28]. RepoBench, RepoCoder, RepoFuse, and GraphCoder instead emphasize repository-level retrieval and completion [29], [30], [31], [32]. ContextBench isolates the context-retrieval behavior of coding agents, while LongCLIBench studies long-horizon command-line programming [33], [34]. These tasks create different repetition patterns and cannot be combined into one unqualified “Code Agent hit rate.”

C. What Must Be Measured in a Trace

The following quantities must remain distinct:

- byte repetition and token repetition;
- longest common token prefix;
- repeated segments outside the prefix;
- reusable tokens under exact execution identity;
- tokens whose reuse is profitable from their current tier;
- task success, which can change under approximate reuse.

Recent agent studies strengthen the motivation but do not replace a target trace. Prompt caching has been evaluated for long-horizon agentic tasks, and trajectory reduction has been proposed to reduce agent cost [35], [36]. Training work also confirms that software-engineering trajectories are long, multi-turn, and expensive to roll out [37], [38]. A project-specific trace must still capture the rendered prompt, tokens, segment boundaries, repository revision, model identity, cache events, and latency breakdown.

IV. KV-CACHE SEMANTICS AND CORRECTNESS

A. Memory Model

For a standard decoder Transformer, a first-order KV footprint is

$$M_{KV} \approx 2LSH_{KV}d_hb, \quad (2)$$

TABLE II
REPRESENTATIVE CODE AGENT WORKLOAD FAMILIES AND CACHE-RESEARCH VALUE.

Family	Representative sources	Dominant interaction	Cache-research value
Issue resolution	SWE-bench, SWE-bench Live, Multi-SWE-bench	Inspect repository, edit multiple files, run tests	Real execution loop; repository/version and test-output changes.
Agent platforms	SWE-agent, OpenHands	Model-directed shell, search, edit, and observation cycle	Exposes adjacent-call reuse and tool-schema stability.
Staged repair	Agentless	Localization, patch generation, validation	Control for separating open-ended agent overhead from task content.
Repository retrieval/completion	RepoBench, RepoCoder, RepoFuse, GraphCoder	Retrieve cross-file context and generate code	Tests deterministic file ordering and non-prefix code repetition.
Trajectory training	SWE-Gym, SWE-smith, long-context RL	Generate or learn from multi-step trajectories	Potential source of public traces and controlled replay workloads.
Long-horizon diagnostics	ContextBench, SWE-Explore, SWE-EVO, LongCLI-Bench	Explore, maintain state, and evolve code across many steps	Measures context churn, compaction, shifted reuse, and tail growth.

where L is the number of layers, S sequence length, H_{KV} stored KV heads, d_h head dimension, and b bytes per element. The factor two accounts for keys and values. Batch size multiplies the result. MQA and GQA reduce H_{KV} ; MLA stores a latent representation plus position-dependent components; sparse and hybrid models may store additional indexes or recurrent state [39], [40], [41].

PagedAttention reduces fragmentation and enables non-contiguous physical allocation [5]. vAttention instead preserves a contiguous virtual layout while allocating physical pages on demand [42]. These are memory-management mechanisms; neither alone guarantees cross-request semantic reuse.

B. Exact-Prefix Reuse Domain

Let requests a and b share a token prefix $x_{1:k}$. Exact reuse is valid only if the execution identity is compatible. A practical cache key must cover at least

$$K = h(M, V, T, A, P, Q, D, x_{1:k}, K_{parent}, \Omega), \quad (3)$$

where M is model family and weights, V model revision, T tokenizer, A adapter or LoRA state, P positional configuration, Q attention and quantization layout, D numerical dtype, K_{parent} the prefix dependency, and Ω the tenant or security domain. Additional execution options may be necessary for a specific engine.

Block-hash chains in vLLM and radix-tree indexing in SGLang encode the dependence on earlier blocks [43], [6]. Prompt Cache adds explicit reusable modules, while ChunkAttention combines a prefix-aware cache with a specialized attention kernel [44], [45]. Exact-prefix reuse should produce the same model state as recomputation, subject to the engine’s numerical determinism and kernel behavior.

C. Why Identical Text May Have Different KV

For a causal Transformer, the hidden state at token i depends on the preceding context. Moving a code block, changing a token before it, altering positions, or changing an adapter can change all downstream states. CacheBlend recovers non-prefix chunks by selectively recomputing influential tokens [9]. EPIC uses position-independent compilation plus boundary repair

[10]. MPIC extends position-independent ideas to multimodal contexts [46]. MEPIC, KV Packet, Irminsul, SparseX, and KVBoost represent a rapidly evolving family with different correction assumptions and architecture constraints [47], [48], [11], [49], [50]. Their results do not justify generic tensor concatenation.

V. REUSE MECHANISMS

A. Exact-Prefix Caching

Exact-prefix systems differ in indexing granularity, scheduling, and physical sharing. PagedAttention supplies page-granular KV memory management; vLLM automatic prefix caching hashes blocks and prefix dependencies [5], [7]. SGLang’s RadixAttention unifies prefix matching, sharing, and eviction within a radix tree [6]. TensorRT-LLM exposes reuse, offload, and priority-based retention controls [8]. Exact matching is the strongest first baseline because it admits a clear no-cache comparison and a strict correctness oracle.

B. Active Prefix Construction

Cache effectiveness is partly a context-builder property. Stable system instructions, tool definitions, and repository rules should appear before volatile content. Serialization, whitespace, file ordering, tool-schema order, and timestamps must be deterministic. Repository content should carry commit or content identifiers. Dynamic tool output and current instructions should be appended where possible. Commercial APIs similarly describe prefix-based reuse and recommend stable common content first [51], [52], [53], [54].

This strategy does not change model semantics if it preserves the intended prompt and ordering constraints. It can nevertheless change agent behavior if information is reordered, so prompt canonicalization requires task-success regression tests rather than token overlap alone.

C. Position-Independent and Partial Reuse

Non-prefix approaches seek reuse when content appears after different prefixes or at shifted positions. Their design space includes precompiling independent modules, correcting positional encodings, recomputing boundary tokens, selecting

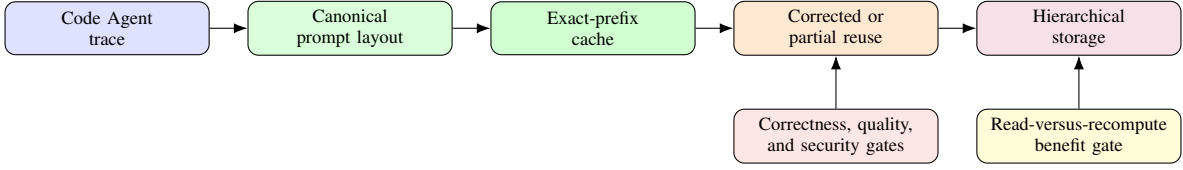


Fig. 2. The survey taxonomy separates workload shaping, semantic reuse, and physical placement. Later mechanisms do not remove earlier correctness gates.

high-deviation tokens, or using an architecture whose stored representation cleanly separates position [44], [10], [11]. The attraction for Code Agents is clear: file order, retrieval rank, compaction, and local edits can invalidate a long exact prefix even when most content repeats. The risk is equally clear: approximation error can alter a tool choice or patch despite small average logit error.

D. Compute, Load, or Combine

The decision need not be binary. Cake overlaps KV loading with local recomputation and adapts to the faster path [55]. This is relevant when SSD or remote bandwidth is variable. A general reuse decision is

$$T_{load} = T_{lookup} + \frac{B_{KV}}{BW_{eff}} + T_{restore}, \quad (4)$$

$$\text{reuse iff } T_{load} + T_{risk} < T_{recompute}, \quad (5)$$

where T_{risk} is zero for an exact compatible hit but may include correction, validation, or an explicit quality penalty for approximate reuse. Equation 5 must be evaluated per tier, size, queue state, and hardware path.

VI. HIERARCHICAL AND DISAGGREGATED KV SYSTEMS

A. Storage Tiers

GPU memory has the lowest access latency but the smallest and most expensive capacity. Host DRAM expands capacity over PCIe or NVLink. NVMe provides much larger capacity with higher latency and sensitivity to request size and layout. Remote memory or storage enables sharing across nodes but adds network and coordination costs. The appropriate object to place is usually a block or chunk with metadata, not an untyped tensor dump.

Mooncake treats distributed KV storage as a first-class serving substrate, trading storage for less computation [13]. MemServe introduces an elastic pool across CPU and GPU memory [12]. LMCache exposes lookup, movement, offload, compression, and cross-engine sharing [14], [56]. Strata focuses on long-context hierarchical caching and fragmented I/O, while HiCache extends RadixAttention beyond GPU memory [15], [16]. Tutti and KVDrive further target SSD-backed and multi-tier operation [57], [58].

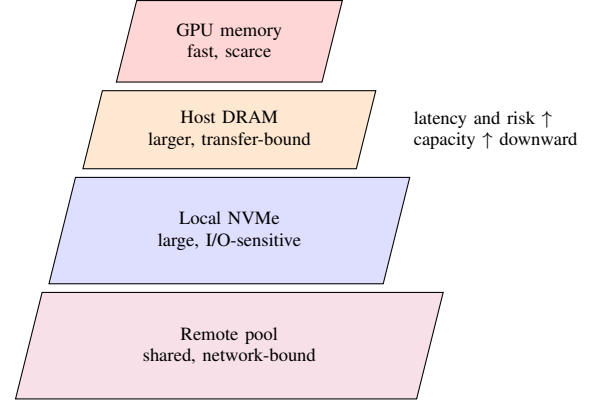


Fig. 3. A logical KV hierarchy. Real ordering depends on interconnect, device, serialization, and access size.

B. Prefill-Decode Disaggregation

DistServe assigns prefill and decode to different GPU pools to reduce interference and independently satisfy TTFT and TPOT objectives [59]. FlowKV emphasizes low-latency KV transfer and load-aware scheduling [60]. Disaggregation increases the importance of transfer layout, network bandwidth, and routing. It should not be conflated with inter-request reuse: a KV object may be transferred once between prefill and decode without ever serving another request.

C. Data Movement and Layout

CacheGen compresses KV for bandwidth-adaptive streaming [61]. InfiniGen dynamically selects and stages KV from host memory, and FlexGen provides a broader single-GPU offloading baseline [62], [63]. Paged GPU layouts may cause small, fragmented transfers; hierarchical systems therefore aggregate blocks, decouple layouts across tiers, pipeline movement with compute, or use GPU-assisted I/O.

VII. COMPRESSION, EVICTION, AND ARCHITECTURE

A. Four Compression Axes

Compression methods change different objects and should not be compared by a single “compression ratio.”

- 1) *Token retention*: H2O, Scissorhands, StreamingLLM, SnapKV, PyramidKV, and DuoAttention retain selected tokens or distinguish attention-head behaviors [64], [65], [66], [67], [68], [69].
- 2) *Numerical precision*: KIVI, KVQuant, QAO, SKVQ, MiniKV, and PQCache use low-bit or product quantization with different granularity and outlier handling [17], [18], [70], [71], [72], [73].

TABLE III
TAXONOMY OF REUSE MECHANISMS FOR CODE AGENT CONTEXTS.

Mechanism	Match condition	Representative work	Main benefit	Principal risk
Exact prefix	Same token prefix and execution identity	vLLM APC, SGLang, ChunkAttention	Deterministic skip of prefill for matched blocks	Fragile to early changes; block tail recomputed.
Structured modules	Declared reusable prompt components	Prompt Cache	Application-controlled modularity	Schema and position constraints; integration burden.
Canonicalized layout	Stable content rendered first and deterministically	Provider guidance; Code Agent context builders	Converts latent repetition into prefix repetition	Reordering can change behavior; requires version discipline.
Corrected non-prefix	Content match plus positional/boundary correction	CacheBlend, EPIC, MEPIC, KVBoost	Reuses shifted or reordered files and retrieved chunks	Approximation, correction cost, model-specific assumptions.
Architecture-native reuse	Representation exposes position-free reusable state	Irmisul for MLA	Lower correction width for compatible MLA implementations	Does not generalize automatically to GQA, hybrid, or opaque runtimes.
Compute-load overlap	Use compute and I/O concurrently	Cake	Robust to variable device or queue performance	Consumes both compute and I/O; scheduling complexity.

TABLE IV
REPRESENTATIVE SYSTEMS AND THE LAYER THEY OPTIMIZE.

System	Primary abstraction	Tiers or topology	Reuse/transfer role	Code Agent relevance
vLLM / PagedAttention	Paged GPU KV blocks	GPU; connectors in current ecosystem	Memory management and exact-prefix APC	Strong no-cache/APC baseline on one node.
SGLang / RadixAttention	Radix tree over shared token prefixes	GPU; extended by HiCache	Exact-prefix lookup, sharing, scheduling, eviction	Natural fit for growing multi-turn histories.
MemServe	Elastic distributed memory pool	GPU and CPU across instances	Inter-request cache plus disaggregated transfer	Evaluates shared context across workers.
Mooncake	KV-centric distributed store	GPU, DRAM, SSD, NIC resources	Persistent distributed cache and scheduling	Reference for scale-out capacity and storage-compute tradeoff.
LMCache	Engine-independent KV layer and connectors	GPU, CPU, local/remote storage	Cross-query offload and cross-engine movement	Useful instrumentation and pluggable prototype layer.
Strata / HiCache	Hierarchical context cache	HBM, host DRAM, disk or external store	Cache-aware loading, prefetch, and scheduling	Directly targets long repeated contexts and deep tiers.
DistServe / FlowKV	Separate prefill and decode workers	GPU pools plus interconnect	Intra-request KV handoff	Relevant when scaling beyond a single RTX 5090 node.
CacheGen	Encoded KV stream	Storage/network to GPU	Transfer compression and adaptation	Tests whether compressed loading beats recompute.
Cake	Concurrent load and compute	GPU plus storage path	Adaptive bidirectional completion	Provides a practical effective-hit decision baseline.

- 3) *Dimension or layer reduction*: MiniCache merges similar cross-layer states and FDC compresses KV dimensions while targeting attention efficiency [74], [75].
- 4) *Transfer encoding*: CacheGen optimizes serialized KV movement rather than only in-GPU attention [61].

These techniques can be complementary, but combinations multiply metadata, conversion, and kernel requirements. Production-oriented evaluation has warned that nominal memory savings can fail to improve throughput and may even increase end-to-end latency [19].

B. Architecture Changes the Cache Object

MQA shares one set of K/V heads across query heads; GQA generalizes this to groups [39], [40]. MLA compresses token state into latent representations and changes how positional components are handled [41], [76]. DeepSeek-V3.2 introduces dynamic sparse attention in a model family that already uses MLA [77]. Hybrid attention or state-space models may alternate full attention, local attention, sparse attention, and recurrent state. A unified cache API is possible, but a single opaque tensor format is not.

At minimum, serialized metadata should include model

and weight revision, layer and attention type, tensor shape and strides, token range, KV-head or latent layout, position encoding, dtype, quantization scales and grouping, sparsity indexes, adapter identity, checksum, and security domain. Compatibility should be declared per model revision and kernel path.

VIII. ADMISSION, PLACEMENT, AND EVICTION

Saving every generated block can waste storage and I/O. A reusable admission score should incorporate reuse probability, recomputation cost, object size, load cost, and quality/security risk. One useful abstraction is

$$V_i = \frac{p_i^{reuse} C_i^{recompute} - C_i^{load} - C_i^{risk}}{B_i \tau_i}, \quad (6)$$

where B_i is stored bytes and τ_i a retention or tier-cost factor. The exact estimator is a research question; Equation 6 prevents a raw hit count from dominating all other objectives.

Placement can be formulated as a constrained decision over GPU, host, SSD, remote, or drop. Hot adjacent-turn prefixes belong near the GPU. Repository content with slower reuse may fit DRAM or SSD. Platform-shared system prompts can be pinned only if the security domain permits sharing. Prefetch should use session, repository, agent step, and queue signals, while preserving a fallback to recomputation.

LRU is an essential baseline but ignores object size and saved compute. Priority-aware retention in current TensorRT-LLM and cache-aware scheduling in SGLang or hierarchical systems provide practical comparison points [8], [16]. Any learned policy must be compared with LRU, size-aware LRU, GreedyDual-style value policies, and an oracle derived from replay.

IX. EVALUATION METHODOLOGY

A. Hit Metrics

Request hit rate alone is weak because a one-block reuse and a nearly complete prefill both count as hits. At least three levels are required:

$$H_{req} = \frac{N(\text{requests with any reuse})}{N(\text{requests})}, \quad (7)$$

$$H_{tok} = \frac{N(\text{reused prefill tokens})}{N(\text{prompt tokens})}, \quad (8)$$

$$H_{eff} = \frac{N(\text{profitably reused tokens})}{N(\text{prompt tokens})}. \quad (9)$$

A token is profitable only if it passes Equation 5. Results should be decomposed by tier, exact versus approximate reuse, tenant scope, context length, and cold/warm state. LMCACHE exposes request- and token-oriented observability concepts that can inform instrumentation [78].

B. Latency, Throughput, Cost, and Quality

The main latency gain is

$$G_{TTFT} = \frac{TTFT_{base} - TTFT_{cache}}{TTFT_{base}}. \quad (10)$$

Report P50, P95, and P99 TTFT, prefill time, queue time, lookup, bytes transferred, restoration, and correction. Decode TPOT must remain visible because prefill scheduling can interfere with decode. Throughput should include requests/s, input tokens/s, output tokens/s, and goodput under explicit SLOs.

End-to-end cost should include GPU time, CPU and DRAM, storage, network, and cache-management overhead:

$$C_{ratio} = \frac{C_{GPU} + C_{CPU} + C_{store} + C_{net} + C_{ops}}{C_{baseline}}. \quad (11)$$

Exact reuse should be checked against a no-cache oracle for logits or generated tokens under controlled determinism. Approximate reuse additionally requires repository-task success, test outcomes, long-context benchmarks, and sensitivity to output length. LongBench and RULER are useful generic controls but cannot replace Code Agent task success [79], [80].

C. Replay Design

A credible experiment needs both real or near-real trajectories and controlled synthetic traces. Real traces establish external validity; synthetic traces isolate mechanisms. Controlled axes should include context length, exact-prefix fraction, shifted repetition, edit location, file order, cache capacity, tier, concurrency, inter-arrival time, and repository version.

The minimum record should contain run and request IDs, timestamp, session, tenant and repository IDs, commit, task class, rendered prompt, token IDs or a privacy-preserving equivalent, segment type and order, model/tokenizer/adapter identity, cache lookup and transfer events, tier, bytes, timing, output, tool action, test result, and file changes. Sensitive source and logs require access control and configurable retention.

D. Hardware Interpretation

The project's current first-node assumption is Ubuntu with one GeForce RTX 5090, 32 GB GDDR7, and 64 GB system memory; a larger HBM-based server remains tentative. The official GPU specification should be recorded rather than describing the 5090's device memory as HBM [81]. This node can support a single-GPU baseline and GPU-to-host experiments, with local NVMe added if specified. It cannot validate multi-node RDMA, shared remote cache, or HBM-cluster claims. Those require a separate environment and should not be inferred from single-node results.

X. SECURITY, PRIVACY, AND ISOLATION

Cross-user cache sharing creates state-dependent timing. Auditing work shows that cache-hit timing can reveal whether a prefix has been cached [82]. SafeKV and CachePrune explore selective sharing or fine-grained privacy-aware reuse [83], [84].

TABLE V
COMPRESSION AND ARCHITECTURE INTERACTIONS.

Class	Reduced dimension	Example methods	Required quality check	Reuse interaction
Token eviction	Sequence positions	H2O, Scissorhands, SnapKV, PyramidKV	Long-context retrieval, code-task success, output length	A cached object is approximate and may be query-dependent.
Low-bit quantization	Bits per K/V element	KIVI, KVQuant, QAA, SKVQ, MiniKV	Logit error, generation agreement, task success	Expands capacity and reduces transfer bytes; adds conversion and scale metadata.
Cross-layer/dimension	Layers or vector width	MiniCache, FDC	Per-model quality and kernel throughput	Format can be incompatible with standard cache connectors.
Transfer encoding	Bytes on storage/network path	CacheGen	End-to-end load latency and reconstruction quality	Useful for lower tiers even if GPU attention remains high precision.
Architectural sharing	Number or form of KV heads/state	MQA, GQA, MLA, DSA/hybrid	Model-native accuracy and runtime compatibility	Changes size formula, position correction, serialization, and potential sharing semantics.

TABLE VI
MINIMUM EVALUATION DIMENSIONS.

Dimension	Required reporting
Correctness	Exact-cache consistency; logit or token agreement; test and task success.
Reuse	Request, token, effective hit rates; prefix/non-prefix; tier; tenant scope.
Latency	Prefill, TTFT P50/P95/P99, lookup, transfer, restore, correction, TPOT.
Capacity	Bytes/token, resident bytes by tier, metadata, fragmentation, compression ratio.
Throughput	Requests/s, input/output tokens/s, goodput under TTFT and TPOT SLOs.
Cost	GPU, CPU, DRAM, storage, network, operations, and per-million-token cost.
Security	Isolation policy, cache-key domain, probe leakage, retention, deletion.

Other work studies direct KV reconstruction, prefix-cache isolation, gateway auditing, and new attack surfaces in position-independent reuse [85], [86], [87], [88].

The safest initial policy is isolation by default: a cache key includes tenant or principal identity, and cross-tenant sharing is limited to explicitly public, immutable content such as platform-controlled instructions. Hash salting, access-control checks, encryption at rest and in transit, deletion semantics, audit logs, and timing analysis are system requirements, not optional additions. A high hit rate does not justify crossing an undefined security boundary.

XI. RESEARCH GAPS FOR CODE AGENTS

A. Public Trace Gap

Agent benchmarks expose tasks and success, but few publish the exact sequence of fully rendered prompts, tokenized segments, cache events, and hardware timing needed for replay. A privacy-conscious Code Agent cache trace format is therefore a useful research artifact in its own right.

B. Context-Builder and Cache Co-Design

Existing serving research usually treats prompts as given. Code Agents actively assemble context from repository search, memory, tool schemas, and compaction. The open question is how to maximize stable prefixes without degrading task quality or starving the model of relevant context. Context retrieval benchmarks and repository methods provide inputs to this co-design but do not optimize KV reuse directly [33], [29], [30].

C. Dependency-Aware Invalidation

When one file changes, exact-prefix systems invalidate all later blocks even if most repository content is unchanged. Arbitrary preservation is unsafe because causal state carries earlier dependencies. Research is needed on dependency-aware layout, explicit module boundaries, positional correction, selective recomputation, and validation that measures tool decisions and code-task success rather than only language-model similarity.

D. Architecture-Portable Cache Objects

GQA, MLA, DSA, multimodal, and hybrid models do not share one physical cache representation. A portable

object model should make incompatibility explicit and support conversion only when correctness, metadata, and kernels are defined. The project term “TensorQuant” does not identify a single public KV format and must remain an unresolved requirement until a specification is provided.

E. Benefit-Aware Hierarchy

The boundary between recomputation and load changes with prompt length, GPU generation, PCIe or NVLink, SSD access size, network congestion, compression, queue depth, and concurrent decode. Static tier thresholds are unlikely to generalize. Online benefit models should be calibrated, compared with Cake-like overlap, and evaluated on tail latency, not just averages.

F. Security-Preserving Reuse

Position-independent and fine-grained sharing increase the number of matchable fragments and potentially the observable cache surface. Security domains, salted indexing, visibility labels, and constant- or noise-shaped response policies must be evaluated together with hit rate.

XII. IMPLICATIONS FOR THE PROPOSED PROJECT

The evidence supports a research sequence, not an immediate full implementation:

- 1) **Trace and measurement first.** Define Code Agent task classes, collect replayable trajectories, and quantify exact prefix, shifted repetition, edit locality, and context churn.
- 2) **Exact baseline second.** Compare no-cache and engine-native exact-prefix caching with identical model, scheduler, and workload settings. Establish correctness and latency decomposition.
- 3) **Context-layout study third.** Test deterministic rendering, stable-first ordering, repository versioning, and controlled compaction. Measure both task success and reusable-token gain.
- 4) **Hierarchy study fourth.** Add host DRAM and then NVMe only after transfer-versus-recompute models are calibrated. Multi-node or HBM-server claims wait for the corresponding hardware.
- 5) **Research mechanisms last.** Study dependency-aware invalidation, position-independent reuse, quantized transport/storage, and architecture portability with explicit quality and security gates.

Table VII maps claims to evidence required. This prevents a logical hit rate from being substituted for end-to-end benefit.

The proposed numerical targets - general hit rate of 70%, Code Agent hit rate of 95%, hit-path TTFT reduction of 90%, and cost ratio no greater than 10% - are not established by the surveyed literature for the target workload. They should remain acceptance hypotheses until the measurement protocol, denominator, trace, and hardware are fixed.

XIII. CONCLUSION

Code Agents are unusually attractive for KV reuse because they make repeated, dependent model calls over stable instructions, tool schemas, histories, and mostly unchanged repositories. The most defensible first mechanism is exact-prefix caching under a complete execution-identity key, strengthened by deterministic context construction. The research frontier begins where exact prefixes break: reordered files, local edits, compaction, heterogeneous cache formats, deep storage tiers, and multi-tenant sharing.

The literature shows that no single hit rate captures this system. Correctness, token coverage, tier location, transfer cost, tail latency, task quality, and isolation must be measured together. Hierarchical storage and compression expand capacity only when their I/O and conversion paths beat recomputation. Non-prefix reuse expands opportunity only when positional and causal dependencies are repaired or proven irrelevant. For the proposed project, the immediate outcome of this survey is therefore a trace-driven research agenda and a disciplined baseline hierarchy, not a claim that ambitious targets have already been met.

CORPUS NOTE

The accompanying research package indexes all 76 papers reviewed here, including additional repository benchmarks, multimodal caching, compression variants, and recent security studies [26], [23], [24], [25], [31], [32], [46], [48], [47], [49], [72], [71], [73]. It also preserves 18 official documentation links and machine-readable metadata, abstracts, BibTeX, licenses, and checksums.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?,” 2023. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [3] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, “Openhands: An open platform for ai software developers as generalist agents,” 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying llm-based software engineering agents,” 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [5] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [6] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “Sglang: Efficient execution of structured language model programs,” 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [7] vLLM Project, “vllm automatic prefix caching,” 2026. Official documentation; accessed 2026-08-30.
- [8] NVIDIA, “Tensortt-llm kv cache reuse,” 2026. Official documentation; accessed 2026-08-30.

TABLE VII
CLAIM-TO-EVIDENCE MATRIX FOR THE PROJECT.

Proposed claim	Required denominator and baseline	Minimum evidence	Invalid shortcut
High Code Agent cache hit rate	All prefill tokens in declared task distribution	Real trace, model/tokenizer identity, tiered H_{tok} and H_{eff}	Request hit rate or text repetition alone.
At least 90% TTFT reduction on hits	Matched no-cache requests under same concurrency	P50/P95/P99 and lookup/transfer/restore breakdown	Comparing warm cache to cold model startup.
Inference cost below 10%	End-to-end cost with cache infrastructure	GPU, CPU, memory, storage, network, and operation accounting	GPU prefill FLOPs alone.
Safe partial reuse	Full recomputation oracle	Logit/token checks plus repository-task and test success	Text similarity or average perplexity only.
Architecture compatibility	Each named model revision and runtime path	Tensor schema, metadata, kernel, quantization, position, and quality tests	Treating every architecture as a standard K/V tensor.
Multi-tenant sharing	Explicit trust and visibility policy	Isolation tests, timing audit, retention/deletion, access logs	Hashing private content without a tenant domain.

- [9] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "Cacheblend: Fast large language model serving for rag with cached knowledge fusion," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [10] J. Hu, W. Huang, W. Wang, H. Wang, T. Hu, Q. Zhang, H. Feng, X. Chen, Y. Shan, and T. Xie, "Epic: Efficient position-independent caching for serving large language models," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [11] B. Ma, J. Eitzinger, and H. Köstler, "Irmisul: Mla-native position-independent caching for agentic llm serving," 2026. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [12] C. Hu, H. Huang, J. Hu, J. Xu, X. Chen, T. Xie, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, "Memserve: Context caching for disaggregated llm serving with elastic memory pool," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [13] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation - a kvcache-centric architecture for serving llm chatbot," in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 155–170, 2025.
- [14] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang, "Lmcache: An efficient kv cache layer for enterprise-scale llm inference," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [15] Z. Xie, Z. Xu, M. Zhao, Y. An, V. S. Malthody, S. Mahlke, M. Garland, and C. Kozyrakis, "Strata: Hierarchical context caching for long context language model serving," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [16] S. Project, "Sglang hicache system design," 2026. Official documentation; accessed 2026-08-30.
- [17] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [18] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [19] W. Gao, X. Zhou, P. Sun, T. Zhang, and Y. Wen, "Rethinking key-value cache compression techniques for large language model serving," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [20] L. Shi, H. Zhang, Y. Yao, Z. Li, and H. Zhao, "Keep the cost down: A review on methods to optimize llm's kv-cache consumption," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [21] H. Li, Y. Li, A. Tian, T. Tang, Z. Xu, X. Chen, N. Hu, W. Dong, Q. Li, and L. Chen, "A survey on large language model acceleration based on kv cache management," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [22] Anthropic, "Claude code overview," 2026. Official documentation; accessed 2026-08-30.
- [23] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "Swe-bench goes live!," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [24] D. Zan, Z. Huang, W. Liu, H. Chen, L. Zhang, S. Xin, L. Chen, Q. Liu, X. Zhong, A. Li, S. Liu, Y. Xiao, L. Chen, Y. Zhang, J. Su, T. Liu, R. Long, K. Shen, and L. Xiang, "Multi-swe-bench: A multilingual benchmark for issue resolving," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [25] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang, "Training software engineering agents and verifiers with swe-gym," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [26] J. Yang, C. E. Jimenez, A. L. Zhang, K. Lieret, J. Yang, X. Wu, O. Press, N. Muennighoff, G. Synnaeve, K. R. Narasimhan, D. Yang, S. I. Wang, and O. Press, "Swe-bench multimodal: Do ai systems generalize to visual software domains?," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [27] S. Zhang, Y. Wang, J. Liang, Y. Shi, W. Zeng, M. Wang, S. He, N. Xu, S. Ye, K. Cai, and X. Gu, "Swe-explore: Benchmarking how coding agents explore repositories," 2026. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-nc-nd/4.0/>.
- [28] T. Le, M. V. T. Thai, D. N. Manh, H. P. Nhat, and N. D. Q. Bui, "Swe-evo: Benchmarking coding agents in long-horizon software evolution scenarios," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [29] T. Liu, C. Xu, and J. McAuley, "Repobench: Benchmarking repository-level code auto-completion systems," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [30] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [31] M. Liang, X. Xie, G. Zhang, X. Zheng, P. Di, wei jiang, H. Chen, C. Wang, and G. Fan, "Repofuse: Repository-level code completion with fused dual context," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-nc-sa/4.0/>.
- [32] W. Liu, A. Yu, D. Zan, B. Shen, W. Zhang, H. Zhao, Z. Jin, and Q. Wang, "Graphcoder: Enhancing repository-level code completion via code context graph-based retrieval and language model," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-nc-sa/4.0/>.
- [33] H. Li, L. Zhu, B. Zhang, R. Feng, J. Wang, Y. Pan, E. T. Barr, F. Sarro, Z. Chu, and H. Ye, "Contextbench: A benchmark for context retrieval in coding agents," 2026. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [34] Y. Feng, J. Sun, Z. Yang, J. Ai, C. Li, Z. Li, F. Zhang, K. He, R. Ma, J. Lin, J. Sun, Y. Xiao, S. Zhou, W. Wu, Y. Liu, P. Liu, Y. Qiao, S. Zhang, and K. Zhang, "Longcli-bench: A preliminary benchmark and study for long-horizon agentic programming in command-line interfaces," 2026.

Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-sa/4.0/>.

- [35] E. Lumer, F. Nizar, A. Jangiti, K. Frank, A. Gulati, M. Phadate, and V. K. Subbiah, "Don't break the cache: An evaluation of prompt caching for long-horizon agentic tasks," 2026. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [36] Y.-A. Xiao, P. Gao, C. Peng, and Y. Xiong, "Reducing cost of llm agents with trajectory reduction," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [37] A. Golubev, M. Trofimova, S. Polezhaev, I. Badertdinov, M. Nekrashevich, A. Shevtsov, S. Karasik, S. Abramov, A. Andriushchenko, F. Fisin, S. Skvortsov, and B. Yangel, "Training long-context, multi-turn software engineering agents with reinforcement learning," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [38] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang, "Swe-smith: Scaling data for software engineering agents," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-sa/4.0/>.
- [39] N. Shazeer, "Fast transformer decoding: One write-head is all you need," 2019. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [40] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," 2023. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [41] DeepSeek-AI, A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Yang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Chen, J. Yuan, J. Qiu, J. Song, K. Dong, K. Gao, K. Guan, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Pan, R. Xu, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Zheng, T. Wang, T. Pei, T. Yuan, T. Sun, W. L. Xiao, W. Zeng, W. An, W. Liu, W. Liang, W. Gao, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Chen, X. Nie, X. Sun, X. Wang, X. Liu, X. Xie, X. Yu, X. Song, X. Zhou, X. Yang, X. Lu, X. Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Zheng, Y. Zhang, Y. Xiong, Y. Zhao, Y. He, Y. Tang, Y. Piao, Y. Dong, Y. Tan, Y. Liu, Y. Wang, Y. Guo, Y. Zhu, Y. Wang, Y. Zou, Y. Zha, Y. Ma, Y. Yan, Y. You, Y. Liu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Huang, Z. Zhang, Z. Xie, Z. Hao, Z. Shao, Z. Wen, Z. Xu, Z. Zhang, Z. Li, Z. Wang, Z. Gu, Z. Li, and Z. Xie, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [42] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, "vattention: Dynamic memory management for serving llms without pagedattention," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [43] vLLM Project, "vllm prefix caching design," 2026. Official documentation; accessed 2026-08-30.
- [44] I. Gim, G. Chen, S. seob Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt cache: Modular attention reuse for low-latency inference," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [45] L. Ye, Z. Tao, Y. Huang, and Y. Li, "Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [46] S. Zhao, J. Hu, R. Huang, J. Zheng, and G. Chen, "Mpic: Position-independent multimodal context caching system for efficient mllm serving," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [47] Q. Wang, Z. Yousefijamarani, M. L. Heisler, R. Gu, B. Xiaolong, S. Yizhou, W. Zhang, W. Lan, Y. Xiong, Y. Zhang, and Z. Fan, "Mepic: Memory efficient position independent caching for llm serving," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-nc-nd/4.0/>.
- [48] C. Chen, G. L. Zhang, X. Yin, C. Zhuo, B. Li, and U. Schlichtmann, "Kv packet: Recomputation-free context-independent kv caching for llms," 2026. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [49] Q. Zhang, K. Chen, N. Liao, Z. Lin, B. Tang, F. Xiong, Z. Li, and X. Wang, "Sparsex: Efficient segment-level kv cache sharing for interleaved llm serving," 2026. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [50] S. Unnikrishnan, "Kvboost: Chunk-level key-value cache reuse with deviation-guided recomputation for efficient large language model inference," 2026. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [51] OpenAI, "Openai prompt caching," 2026. Official documentation; accessed 2026-08-30.
- [52] Anthropic, "Anthropic prompt caching," 2026. Official documentation; accessed 2026-08-30.
- [53] Google, "Google gemini context caching," 2026. Official documentation; accessed 2026-08-30.
- [54] DeepSeek, "Deepseek context caching," 2026. Official documentation; accessed 2026-08-30.
- [55] S. Jin, X. Liu, Q. Zhang, and Z. M. Mao, "Compute or load kv cache? why not both?," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [56] L. Project, "Lmcache documentation," 2026. Official documentation; accessed 2026-08-30.
- [57] S. Qiu, Y. Hu, X. Wang, W. Zhu, J. Yan, H. Chen, K. Xu, K. Chen, and Y. Zhang, "Tutti: Making ssd-backed kv cache practical for long-context llm serving," 2026. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [58] J. Lin, J. Mi, Z. Hong, H. Wang, Q. Liu, H. Zhang, P. Li, and S. Guo, "Kvdrive: A holistic multi-tier kv cache management system for long-context llm inference," 2026. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [59] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-sa/4.0/>.
- [60] W. Li, G. Jiang, X. Ding, Z. Tao, C. Hao, C. Xu, Y. Zhang, and H. Wang, "Flowkv: A disaggregated inference framework with low-latency kv cache transfer and load-aware scheduling," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [61] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "Cachegen: Kv cache compression and streaming for fast large language model serving," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [62] W. Lee, J. Lee, J. Seo, and J. Sim, "Infinigen: Efficient generative inference of large language models with dynamic kv cache management," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [63] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [64] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H₂o: Heavy-hitter oracle for efficient generative inference of large language models," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [65] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [66] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," 2023. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [67] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [68] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu, and W. Xiao, "Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [69] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han, "Duoattention: Efficient long-context llm inference with retrieval and streaming heads," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.

- [70] S. Dong, W. Cheng, J. Qin, and W. Wang, "Qaq: Quality adaptive quantization for llm kv cache," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by-nc-sa/4.0/>.
- [71] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, and D. Lin, "Skvq: Sliding-window key and value cache quantization for large language models," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [72] A. Sharma, H. Ding, J. Li, N. Dani, and M. Zhang, "Minikv: Pushing the limits of llm inference via 2-bit layer-discriminative kv cache," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [73] H. Zhang, X. Ji, Y. Chen, F. Fu, X. Miao, X. Nie, W. Chen, and B. Cui, "Pqcache: Product quantization-based kvcache for long context llm inference," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [74] A. Liu, J. Liu, Z. Pan, Y. He, G. Haffari, and B. Zhuang, "Minicache: Kv cache compression in depth dimension for large language models," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [75] Z. Zhang and H. Shen, "Fdc: Fast kv dimensionality compression for efficient llm inference," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [76] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Sun, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Xiong, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2024. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [77] DeepSeek-AI, A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, C. Lu, C. Zhao, C. Deng, C. Xu, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, E. Li, F. Zhou, F. Lin, F. Dai, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Li, H. Liang, H. Wei, H. Zhang, H. Luo, H. Ji, H. Ding, H. Tang, H. Cao, H. Gao, H. Qu, H. Zeng, J. Huang, J. Li, J. Xu, J. Hu, J. Chen, J. Xiang, J. Yuan, J. Cheng, J. Zhu, J. Ran, J. Jiang, J. Qiu, J. Li, J. Song, K. Dong, K. Gao, K. Guan, K. Huang, K. Zhou, K. Huang, K. Yu, L. Wang, L. Zhang, L. Wang, L. Zhao, L. Yin, L. Guo, L. Luo, L. Ma, L. Wang, L. Zhang, M. S. Di, M. Y. Xu, M. Zhang, M. Zhang, M. Tang, M. Zhou, P. Huang, P. Cong, P. Wang, Q. Wang, Q. Zhu, Q. Li, Q. Chen, Q. Du, R. Xu, R. Ge, R. Zhang, R. Pan, R. Wang, R. Yin, R. Xu, R. Shen, R. Zhang, S. H. Liu, S. Lu, S. Zhou, S. Chen, S. Cai, S. Chen, S. Hu, S. Liu, S. Hu, S. Ma, S. Wang, S. Yu, S. Zhou, S. Pan, S. Zhou, T. Ni, T. Yun, T. Pei, T. Ye, T. Yue, W. Zeng, W. Liu, W. Liang, W. Pang, W. Luo, W. Gao, W. Zhang, X. Gao, X. Wang, X. Bi, X. Liu, X. Wang, X. Chen, X. Zhang, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Li, X. Yang, X. Li, X. Chen, X. Su, X. Pan, X. Lin, X. Fu, Y. Q. Wang, Y. Zhang, Y. Xu, Y. Ma, Y. Li, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Qian, Y. Yu, Y. Zhang, Y. Ding, Y. Shi, Y. Xiong, Y. He, Y. Zhou, Y. Zhong, Y. Piao, Y. Wang, Y. Chen, Y. Tan, Y. Wei, Y. Ma, Y. Liu, Y. Yang, Y. Guo, Y. Wu, Y. Wu, Y. Cheng, Y. Ou, Y. Xu, Y. Wang, Y. Gong, Y. Wu, Y. Zou, Y. Li, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Zhao, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Huang, Z. Wu, Z. Li,
- Z. Zhang, Z. Xu, Z. Wang, Z. Gu, Z. Zhu, Z. Li, Z. Zhang, Z. Xie, Z. Gao, Z. Pan, Z. Yao, B. Feng, H. Li, J. L. Cai, J. Ni, L. Xu, M. Li, N. Tian, R. J. Chen, R. L. Jin, S. S. Li, S. Zhou, T. Sun, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Song, X. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Z. Huang, Z. Xu, Z. Zhang, D. Ji, J. Liang, J. Guo, J. Chen, L. Xia, M. Wang, M. Li, P. Zhang, R. Chen, S. Sun, S. Wu, S. Ye, T. Wang, W. L. Xiao, W. An, X. Wang, X. Sun, X. Wang, Y. Tang, Y. Zha, Z. Zhang, Z. Ju, Z. Zhang, and Z. Qu, "Deepseek-v3.2: Pushing the frontier of open large language models," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [78] L. Project, "Lmcache metrics reference," 2026. Official documentation; accessed 2026-08-30.
- [79] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench: A bilingual, multitask benchmark for long context understanding," 2023. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [80] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekes, F. Jia, Y. Zhang, and B. Ginsburg, "Ruler: What's the real context size of your long-context language models?," 2024. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [81] NVIDIA, "Nvidia geforce rtx 5090 specifications," 2026. Official documentation; accessed 2026-08-30.
- [82] C. Gu, X. L. Li, R. Kudipudi, P. Liang, and T. Hashimoto, "Auditing prompt caching in language model apis," 2025. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [83] K. Chu, Z. Lin, D. Xiang, Z. Shen, J. Su, C. Chu, Y. Yang, W. Zhang, W. Wu, and W. Zhang, "Selective kv-cache sharing to mitigate timing side-channels in llm inference," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [84] G. Wu, Z. li, Y. Zhang, Z. Zhang, J. Niu, Y. Wu, and Y. Zhang, "Cacheprune: Privacy-aware and fine-grained kv cache sharing for efficient llm inference," 2026. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.
- [85] Z. Luo, S. Shao, S. Zhang, L. Zhou, Y. Hu, C. Zhao, Z. Liu, and Z. Qin, "Shadow in the cache: Unveiling and mitigating privacy risks of kv-cache in llm inference," 2025. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [86] P. G. Pennas, K. Papaioannou, M. Guarnieri, and T. D. Doudali, "Prefixwall: Mitigating prefix caching side channels in shared llm systems," 2026. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [87] R. Fahey, "Cacheprobe: Auditing prompt cache isolation in gateway apis," 2026. Accessed 2026-08-30; license <http://arxiv.org/licenses/nonexclusive-distrib/1.0/>.
- [88] Y. Zhang, Z. Wang, H. Zhang, and Y. Yang, "Hijackkv: New threat in position-independent kv cache reuse," 2026. Accessed 2026-08-30; license <https://creativecommons.org/licenses/by/4.0/>.